

Huffman-ov algoritam

HUFFMAN(W,e)
INIT-QUEUE(H,e)

Ulaz u algoritam predstavlja **brojEksternihČvorova(e)** i njihove pridružene težine date u vektoru **W[1:e]**, a algoritam izgrađuje stablo sa minimalnom težinskom eksternom dužinom puta i vraća adresu njegovog korena .

for i=1 to e do
z = GETNODE
w(z) = W[i]
PQ-INSERT(H,z)
end_for

Na početku, algoritam inicijalizuje prioritetni red H, alocira prostor za **e** eksternih čvorova, upisuje njihove težine u polje **w** i ispisuje ih u prioritetni red **H** koji je organizovan po rastućim vrednostima polja **w**.

Polje **w** u eksternim čvorovima ovog stabla sadrži njihovu težinu, a u ostalim čvorovima predstavlja zbir težina svih eksternih čvorova koji su potomci tog čvora.

Ovaj gore prikazani dio algoritma nam služi da napravimo ŠUMU. Evo kako to funkcioniše:

1. *HUFFMAN(W,e)* znači da našoj funkciji kao argumente redom prosleđujemo jednodimenzionalni vektor **W[1:e]** – tako da su duž tog vektora pohranjene težine (npr.za prvi primjer sa vježbi A8, B6 ... imamo **W[0] = 8, W[1] = 6...**) i broj nosača informacija – broj eksternih čvorova.

2. *INIT-QUEUE(H,e)* je naredba koja dinamički alocira prostor za skladištenje **e** čvorova u prioritetni red-sortiranaDvostrukoPOvezanaLista **H**.

3.

z = GETNODE

w(z) = W[i]

Iz dinamičke zone memorije, dohvaćamo skladište za skladištenje jednog čvora, a zatim sa i-te pozicije skladišta težina **W**-vector očitamo vrijednost uskladištenu u **W[i]** te funkcijom **w(z)** našem novom čvoru pridružimo tu pročitano eksternu težinu.

4. Čvor ubacimo u prioritetni red **H** naredbom *PQ-INSERT(H,z)*, a kako je red prioritetan -dvostruko-povezana-uređena-lista taj čvor automatski ide na parvo mjesto tako da se održava rastući poredak u listi.

IMAMO ŠUMU. Dalje algoritam ide da za tu šumu oformira stablo.

for i=1 to e-1 do
z = GETNODE
x = PQ-MIN-DELETE(H)
y = PQ-MIN-DELETE(H)
w(z) = w(x) + w(y)
left(z) = x
right(z) = y
PQ-INSERT(H, z)
end_for
return z

U svakoj iteraciji algoritma se formira jedan čvor grananja čija je težina jednaka zbiru težina svih eksternih čvorova koji su njegovi potomci, a kako je broj čvorova grananja za jedan manji od broja eksternih potrebno je **e-1** iteracija. Tako se u prvoj iteraciji iz reda **H** izvlače dva čvora sa najmanjim težinama **x** i **y**, napravi novi čvor **z** sa težinom koja je jednaka njihovom zbiru pa **z** postaje koren stabla u kojem su **x** i **y** koreni levog i desnog podstabla. Koren novog stabla se zatim ubaci u prioritetni red **H**.

BFS

BFS(G,v)

Funkcija kao argument prima graf(graph G) i početni čvor (vertex v).

for $i = 1$ **to** n **do**
 $visit[i] = false$
end_for

Od tog početnog-prvog čvora ($i=1$) pa sve do poslednjeg čvora u grafu ,a imamo ih n -stepenGrafa svaku ćeliju visit-vector-a ažuriramo na 0. Završavamo sa ažuriranjem.

$visit[v] = true$

Dakle, posjećujemo čvor, ažiramo ćeliju visit vector-a za taj čvor. Ažuriramo sa false na true tako da se ubuduće zna da je taj čvor već posjećen.

$insert(Q,v)$

Taj čvor koji posjećujemo stavljamo u neprioritetni red Q.
U budućnosti, tj. tokom daljeg rada algoritma iz tog reda se izbacuje čvor te se u taj red ubacuju NEPSOJEĆENI susjedni čvorovi tog izbačenog čvora.
Konkretno ovdje u skladu sa radom algoritma, čvor koji je argument funkcije se ubacuje u red.

while(not $QUEUE-EMPTY(Q)$) **do**

Dakle, algoritam će raditi sve dok se ima šta ažurirati za taj neprioritetni red Q. Prazan(Q) <---> završen posao!

$v = delete(Q)$

Uzimamo čvor – skidamo ga (FIFO) – iz prioritetnog reda.
Dakle, v je sada čvor iz Grafa.

for $\{ u : (v,u) \in E \}$ **do**

Pazi, petljom... Dakle susjed po susjed čvora v, a to je svaki čvor u koji takav između njega i čvora v postoji neka binarna relacija – grana grafa E.

if(not $visit[u]$) **then**

E ako taj čvor u do sada nije posjećivan, onda ga moramo posjetiti-obrađiti što znači da idemo u tijelo ovoga if-a:
Evidentiramo posjećenost – ažuriramo visit-vector te taj čvor u ubacujemo u neprioritetni red Q kako bi kasnije mogli izbaciti isti iz reda, te po potrebi obići njegove susjede...

$visit[u] = true$
 $INSERT(Q, u)$
end_if
end_for
end_while

DFS

DFS(G, v)

Funkcija kao argument prihvaća graf G i početni čvor v tog grafa, čvor počevši od kojeg pa na dalje obilazimo naš graf.

```
for  $i=1$  to  $n$  do  
     $visit[i] = false$   
end_for  
DFS_VISIT(v)
```

Algoritam počinje tako što se za svaki čvor grafa ažurira visit-vector. Na početku, svaki čvor se okarakteriše kao neposjećen.

Smatramo da je *DFS_VISIT* već definisana funkcija. Pozivamo istu da obavi daljnji posao.

```
DFS-VISIT(v)  
 $visit[v] = true$ 
```

Pomoćna funkcija kao argument prihvata čvor grafa, ažurira njegovu posjećenost...

```
for  $\{ u, (v,u) \in E \}$  do  
    if (not  $visit[u]$ ) then  
        DFS-VISIT(u)  
    end_if  
end_for
```

Za taj argument čvor idemo dubinom susjed po susjed...
Te ako neki susjed nije posjećen odma slijedi poziv pomoćne funkcije tj.

Prim-ov algoritam

Prim(G, s)

Algoritam, tj. funkcija kao argument prihvata graf G i proizvoljno odabran čvor s iz tog grafa. Taj čvor s će postati koren novonastalog MST(minimal spanning tree).

$U = \{ s \}$

$E' = \text{prazanSkup}$

U je skup čvorova. Po završetku algoritma treba da bude $U = V$.

To je skup koji je podskup skupa E-edges grana grafa G. Dakle skup E' će sadržati samo odabrane grane zadatog grafa G.

while ($U \neq V$) **do**

Sve dok se skup U ne dopuni do skupa V algoritam radi i MST gradi. Dakle, čvor po čvor, trazi se najkraće rastojanje, povezuju se čvorovi...

$find(u, v) \Rightarrow \min \{ w_{(u,v)} : (u \text{ pripada } U) \text{ and } (v \text{ pripada } (V - U)) \}$

Za uređeni par čvorova (u_{iz_U} i v_{iz_V}) pronađi granu najmanje težine tj. $w(u,v)$, pazi još ovo u pripada skupu U, a v je element pripada skupu V, a da takvog nema u skupu U.

$U = U + \{v\}$

$E' = E' + \{(u,v)\}$

end_while

Proširi skup čvorova, tj v dodaj u skup U, takođe treba proširiti skup E' pa ćemo u taj skup E' dodati ovu granu koju smo gore ponašli. To je grana iz skupa E takva da je ona najmanje težine od svih grana koje povezuju naše pomenute čvorove u i v.

$MST = (U, E')$

Formiranje MST-a.

Kruskal-ov algoritam

Algoritam polazi od šume $(T_1, T_2, \dots, T_n)$ koja se sastoji od nepovezanih čvorova grafa bez ijedne grane, a zatim dodaje po jednu granu dok se šuma ne pretvori u stablo za koje se garantuje da je minimalno obuhvatno stablo grafa G.

Kruskal(G) Kao argument funkcija prihvaća graf G .

$E' = \{ \text{prazan skup} \}$ Na početku, skup grana MST-a koje će biti formirano je prazan.

for each (u, v) <i>iz</i> E do $PQ\text{-}INSERT(PQ, w(u,v))$ end for	Sve grane iz skupa E , tj. skupa svih grana grafa G u neopadajućem redoslijedu se skladište u prioritetni red PQ .
--	--

$num = 0$ Pomoćni indeks koji će detektovati signal za pestanak rada algoritma.

while(num < n-1) **do** Algoritam se završava kada se uključi n-1 grana i tada su svi čvorovi u jednoj povezanoj komponenti, koja predstavlja traženo minimalno minimalno obuhvatno stablo. Dok se to ne postigne algoritam radi (do):

$w(u,v) = PQ-MIN-DELETE(PQ)$ par čvorova (u,v) poveži granom najmanje težine ...

if(*(u pripada T_i) and (v pripada T_j) and ($i \neq j$)*) **then** uslov je da su u i v iz disjunktnih skupova, ne
smije se formirati zatvorena putanja!

$E' = E' + \{(u, v)\}$	Prethodni uslov je zadovoljen! Tu granu iz E dodaj u E'.
$T_k = T_i + T_j$	Evidentiraj poništavanje disjunktnosti – ažuriraj Šumu..
$num = num + 1$	Signaliziraj uspjeh.

end_if

end_while
$$MST(V\text{-vertex}, E')$$

WARSHALL

WARSHALL(A)

A – matrica netežinskog grafa G.

$P = A$

Inicijalizacija matrice puta tj. matrice P.

for $k=1$ to n **do**

for $i=1$ to n **do**

for $j=1$ to n **do**

$p[i,j] = p[i,j]$ or $(p[i,k]$ and $p[k,j])$

end_for

end_for

end_for

Floyd

FLOYD(W) Ovaj algoritam koristi kao ulaz matricu težina grana W reda $n \times n$ definisanu kao

$w[i,j] = 0$	ako je $i=j$
$w[i,j] = w(i,j)$	ako je $i \neq j$ i (i,j) pripada E
$w[i,j] = +\text{infinity}$	ako je $i \neq j$ i (i,j) ne pripada E

$D = W$ Algoritam daje izlaz u obliku matrice D , gde $d[i,j]$ predstavlja najkraće rastojanje između čvorova i i j .

```
for k=1 to n do
  for i=1 to n do
    for j=1 to n do
      if(  $d[i,j] > d[i,k] + d[k,j]$  ) then
         $t[i,j] = t[k,j]$ 
         $d[i,j] = d[i,k] + d[k,j]$ 
      end_if
    end_for
  end_for
end_for
```

DJIKSTRA

DJIKSTRA(W)

Ovaj algoritam koristi kao ulaz matricu težina grana W reda $n \times n$ definisanu kao

$w[i,j] = 0$ ako je $i=j$,
 $w[i,j] = w(i,j)$ ako je $i \neq j$ i (i,j) pripada E ,
 $w[i,j] = +\infty$ ako je $i \neq j$ i (i,j) ne pripada E .

U ovom slučaju pretpostavlja se da su težine grana nenegativne, što je od suštinske važnosti za rad algoritma.

Neka su čvorovi numerisani sa 1, 2, ..., n , a sa 1 je označen polazni čvor od kojeg se traže najkraća rastojanja do svih ostalih čvorova.

Određivanje najkraćih puteva od jednog čvora do svih ostalih je problem koji se može riješiti primjenom Dijkstra-inovog algoritma. Ulaz predstavlja matrica težina W , a izlaz čine vektori d i t od po n članova, gdje je $d[i]$ najkraće rastojanje od polaznog čvora 1 do čvora i , a $t[i]$ je čvor prethodnik čvora i na najkraćem putu od čvora 1.

$S = \{ 1 \}$

U skupu S se pamte čvorovi za koje su do tog trenutka nađena apsolutna najkraća rastojanja od polaznog čvora. U skupu $V-S$ se nalaze preostali čvorovi za koje postoje trenutne procjene najkraćih rastojanja od čvora 1 koje se eventualno kasnije mogu skratiti.

```
for  $i=2$  to  $n$  do  
   $d[i] = w[1,i]$   
  if ( $w[1,i] \neq \infty$ ) then  
     $t[i] = 1$   
  else  
     $t[i] = 0$   
  end_if
```

```
for  $k=1$  to  $n-1$  do
```

```
  find  $\min\{ d[i] : i \text{ pripada } (V-S) \}$ 
```

```
   $S = S + \{ i \}$ 
```

```
  for each  $j$  iz Skupa  $(V-S)$  do
```

```
    if ( $d[i] + w[i,j] < d[j]$ ) then
```

```
       $d[j] = d[i] + w[i,j]$ 
```

```
       $t[j] = i$ 
```

```
    end_if
```

```
  end_for
```

```
end_for
```